

LINUX_TOMORROW_QUICKSTART

NetworkPeer — Linux Laptop Quickstart (Path B: full cloud deploy)

Status (verified Aug 11 ~06:20 on the Mac): backend `typecheck` / `lint` / `build` , 21 unit tests , funded atomic-acceptance E2E  (create → fund → signed webhook → POSTED → two workers race → exactly one 200 / one 409), live boot  (`/live` , `/health` → database + postgis true), real OTP → login → protected route , frontend dev on 8080 , **frontend** `NITRO_PRESET=vercel` **production build** , **backend Docker image builds AND boots with demo-mode env** , S3 `networkpeer-v1` reachable (versioning ; **IAM inline policy created + `verify-s3-setup.mjs` all green**  — policy name is arbitrary; the one used here is named `new`).

Path A (Mac tunnel) is dead — the Mac will NOT be in the office. This quickstart is the Path B plan: everything in the cloud, the Linux laptop only opens URLs. The detailed runbook is `docs/DEPLOY_CLOUD_LINUX_ONLY.md` — follow it step by step; this page is the summary + tonight's action list + the shipping checklist.

Step 1 — Tonight on the Mac (2–4 h total, do these three in parallel where possible)

1a. S3 IAM fix (DONE — created as inline policy named `new`)

Where: AWS Console → IAM → Users → `networkpeer-demo` → Permissions → **Add permissions** → **Create inline policy** → **JSON** → paste → Next → Create policy. (No aws CLI on the Mac and this user can't edit its own policy, so

console-only. Bucket: `networkpeer-v1` .) The policy **name is arbitrary** — nothing in the code reads it, only the attached actions matter. It was saved here as `new` .

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:PutObjectTagging",
        "s3:GetBucketVersioning",
        "s3:GetBucketPublicAccessBlock",
        "s3:GetEncryptionConfiguration",
        "s3:PutBucketCORS"
      ],
      "Resource": ["arn:aws:s3:::networkpeer-v1",
        "arn:aws:s3:::networkpeer-v1/*"]
    }
  ]
}
```

Verify (DONE cd

`/Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main && node scripts/verify-s3-setup.mjs` → all  (also sets CORS for `localhost:8080`).

After the Vercel deploy (Step 1c), re-run with the Vercel origin so browser uploads work from the live site — the script accepts extra origins and re-sets CORS:

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main &&
node scripts/verify-s3-setup.mjs https://<vercel-origin>
```

1b. Railway (15 min) — database, Redis, API

Follow `DEPLOY_CLOUD_LINUX_ONLY.md` **Steps 1–2**: create Postgres + Redis plugins, run the migrations + role provisioning from the Mac (`npm run migrate + provision-app-role.sql` against the Railway `DATABASE_URL` with `?sslmode=require`), then deploy the API service (Root directory `NetworkPeer-main` , Dockerfile build, the full demo-mode Variables block from Step 2 — `NODE_ENV=development` , `OTP_ECHO_IN_RESPONSE=true` , `SMS_PROVIDER=console` , `PAYMENT_GATEWAY=stub` , `PLATFORM_FEE_BPS=0`). Verify `https://<api>.up.railway.app/api/v1/live` and `/api/v1/health` → `database: true` .

1c. Vercel (10 min) — frontend

Follow `DEPLOY_CLOUD_LINUX_ONLY.md` **Step 3**: import the repo, Root directory `NetworkPeer-platform-main`, Node 24, build `npm run build`, env `NITRO_PRESET=vercel`, `VITE_API_BASE_URL=https://<api>.up.railway.app/api/v1`, `VITE_API_PREFIX=/api/v1`, `VITE_DEMO_WEBHOOK_SECRET=<same as backend>`. After it deploys, copy the Vercel origin into the backend's `CORS_ORIGINS` and **redploy the API**.

1d. Rehearse from the Mac (15 min)

Open the **Vercel URL** in a normal + incognito window and run Step 3 below end-to-end (funding via `https://<vercel-origin>/dev/settle-funding`, not the local simulator). If it works here it works identically on the Linux laptop.

Step 2 — Tomorrow on the Linux laptop (browser only, 5 min)

1. Open the **Vercel URL** in a normal window (this is the *client*).
2. Open the **same URL in an incognito/private window** (this is the *worker*).
3. That's the whole install. No Node, no Docker, no accounts, no code.

Corporate network: needs HTTPS outbound (normally allowed). If blocked, use a phone hotspot on the Linux laptop — the app is fully cloud-hosted, so it works from any network.

Step 3 — The boss demo (10 min, identical on Mac and Linux)

1. **Home** — animated gradient wordmark, drifting blobs, 3D-tilt demo card (hover it), scroll-reveal sections.
2. **Sign in** — client in the normal window, worker in incognito (`/auth`, role Worker). **OTP appears on the verify screen** (demo mode — console OTP echo; Twilio was skipped because it needs a paid plan). Any phone number works.
3. **Verify the worker** (one time, from the Mac with `psql` against the **cloud** DB — not the local Docker one):

```

psql "postgresql://postgres:PASSWORD@host.up.railway.app:PORT/railway?
      sslmode=require" -f NetworkPeer-main/scripts/provision-admin.sql
psql "postgresql://postgres:PASSWORD@host.up.railway.app:PORT/railway?
      sslmode=require" \
  --set=admin_phone="+1CLIENT-PHONE" --
      set=worker_phone="+1WORKER-PHONE" <<'SQL '
SELECT public.admin_set_worker_verification(
  (SELECT id FROM public.users WHERE phone_number =
    : 'admin_phone'),
  (SELECT id FROM public.users WHERE phone_number =
    : 'worker_phone'),
  'VERIFIED', TRUE, 'Verified for live demonstration');
SQL

```

4. **Client dashboard** — five stat cards with large labels/numbers.
5. **Create a job** (/client/jobs/new) — checklist count 3 → three task boxes appear; map-pick a location (search “Connaught Place, New Delhi” or tap the map); budget 500 ; **Post job** (status FUNDING , hidden from workers).
6. **Fund escrow** — click it, copy operationId + providerReference , open <https://<vercel-origin>/dev/settle-funding> , paste both, click **Settle funding** → job becomes **POSTED** (this is what Stripe’s webhook does).
7. **Worker accepts** (incognito) — nearby list shows it (client identity hidden) → Accept → private details. Optional wow-factor: a second incognito window accepting simultaneously — exactly one succeeds.
8. **Live task** — EN ROUTE → AT LOCATION → IN PROGRESS.
9. **Evidence** — capture/select photos → uploads straight to S3 → confirmed → **Submit** (needs §1a).
10. **Client approves** → worker wallet shows full payout, **no platform fee**.
11. **Realtime** — with both windows open, watch the notification bell + toast in the other.

Step 4 — Files to ship to the office laptop (final verdict)

Email or Teams-message **these three things** (nothing else — no code, no repos):

1. docs/EXECUTIVE_PRESENTATION_AND_DEPLOYMENT_GUIDE.docx — demo script + talking points (.md version too if email accepts it).

2. docs/NETWORKPEER_COMPLETE_GUIDE.docx — full technical breakdown for hard questions.
3. **The Vercel URL** (the app) — open in normal + incognito windows, plus the note that funding settlement happens at `https://<vercel-origin>/dev/settle-funding`.

Optional: this quickstart (`.md` / `.docx`) so they can follow the demo script themselves.

Never ship secrets: AWS keys, `.env`, and the Railway `DATABASE_URL` stay in the cloud platform's env vars — the office laptop only ever opens URLs. Twilio is NOT in this demo.

Step 5 — Seeing the data (how the DB is visible)

- **Cloud (Railway):** service → **Data** tab → click a table (e.g. `jobs`) to browse rows, or **Query** for SQL. From the Mac: `psql "<railway-database-url>" → \dt, \d jobs, SELECT * FROM schema_migrations ORDER BY filename DESC LIMIT 3;` . Full section: `DEPLOY_CLOUD_LINUX_ONLY.md` **Step 1.5**.
 - **Local Mac tonight:** Docker PostGIS at `localhost:5433` — `docker exec networkpeer-postgis psql -U postgres -d networkpeer, npm run db:shell`, or any Postgres GUI.
-

Quick reference

```
./scripts/start-demo.sh      # local Mac rehearsal (no
                             tunnels needed for Path B)
./scripts/stop-demo.sh      # stop local demo
node scripts/verify-s3-setup.mjs  # S3 readiness + CORS (see
                             $1a)
psql "<railway-database-url>"  # cloud DB shell
                             (provisioning + inspection)
# Settlement during the cloud demo: https://<vercel-
                             origin>/dev/settle-funding
```

Known limits to be honest about: admin screens are prototypes; browser push (FCM) isn't wired; realtime toasts may not fire on Vercel serverless (statuses still update on refresh); OTP is console-echoed on the verify screen (Twilio skipped — needs a paid plan).